Karsten Keßler

Gradient und Niveaulinien

Stand: 19.10.2026

tutor.kkessler.de

Gradient, Niveaulinien und Gradientenverfahren

1. Gradient und Niveaumengen
2. Der Gradient steht senkrecht auf der Niveaumenge
3. Richtung des steilsten Anstiegs
4. Das Gradientenverfahren
5. Wahl der Schrittweite
6. Übungsaufgaben
7. Ausblick: Gradientenverfahren in der KI

1. Gradient und Niveaumengen

Sei $D \subseteq \mathbb{R}^n$ offen und $f: D \to \mathbb{R}$ stetig partiell differenzierbar ($f \in C^1(D)$). Der Gradient fasst die partiellen Ableitungen zu einem Vektor zusammen:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x), \dots, \frac{\partial f}{\partial x_n}(x) \right)^{\mathrm{T}}$$

Definition: Niveaumenge

Für $c \in \mathbb{R}$ heißt $N_c = \{x \in D : f(x) = c\}$ die Niveaumenge von f zum Niveau c. Im $\mathbb{R}^2$ heißt N_c Höhenlinie (Niveaulinie), im $\mathbb{R}^3$ Niveaufläche.

Auf einer Höhenlinie bleibt der Funktionswert konstant, wie bei den Höhenlinien einer Wanderkarte. Der Gradient zeigt dagegen in die Richtung, in der f am schnellsten wächst. Die folgenden Seiten zeigen, dass diese beiden Richtungen stets senkrecht aufeinander stehen.

Beispiel: $f(x, y) = x^2 + 4y^2$

$$\nabla f(x, y) = (2x, 8y)^{\mathrm{T}}$$

Höhenlinie N_c: $x^2 + 4y^2 = c$, für $c > 0$ eine Ellipse mit den Halbachsen $\sqrt{c}$ und $^1/_2\sqrt{c}$

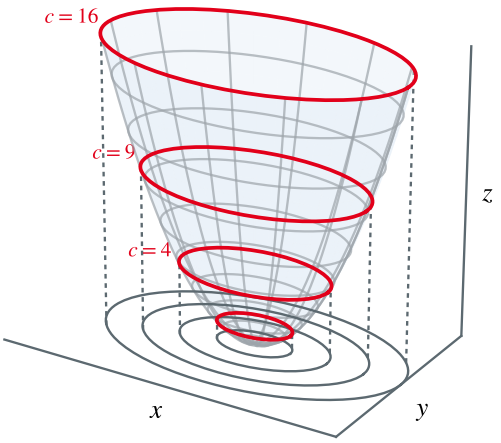


Abb. 1: Fläche $z = f(x, y)$, Schnitt in Höhe c (rot) und Projektion in die xy-Ebene

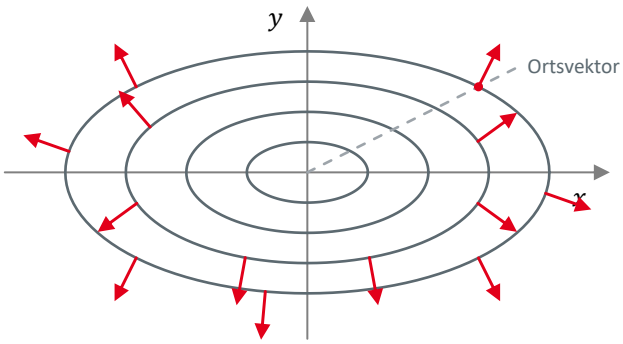


Abb. 2: Draufsicht: Höhenlinien $c = 1, 4, 9, 16$ mit Gradientenpfeilen (normiert)

Abb. 2 ist die Draufsicht auf Abb. 1. Bei Kreisen $f = x^2 + y^2$ zeigt $\nabla f = (2x, 2y)^{\mathrm{T}}$ radial nach außen. Bei Ellipsen nicht mehr: Im markierten Punkt weicht der Pfeil von der gestrichelten Richtung des Ortsvektors ab. Senkrecht zur Höhenlinie steht er trotzdem.

2. Der Gradient steht senkrecht auf der Niveaumenge

Satz (Gradient und Niveaumenge)

Sei $D \subseteq \mathbb{R}^n$ offen, $f \in C^1(D)$, $c \in \mathbb{R}$ und $x_0 \in N_c$. Ist $\gamma: I \to D$ ($I \subseteq \mathbb{R}$ ein Intervall) eine differenzierbare Kurve, die ganz in der Niveaumenge verläuft, d. h. $f(\gamma(t)) = c$ für alle $t \in I$, und gilt $\gamma(t_0) = x_0$, so ist

$$\nabla f(x_0) \cdot \gamma'(t_0) = 0.$$

Der Gradient steht also senkrecht auf jedem Tangentenvektor an N_c in x_0. Für $\nabla f(x_0) \neq 0$ ist $\nabla f(x_0)$ ein Normalenvektor der Niveaumenge.

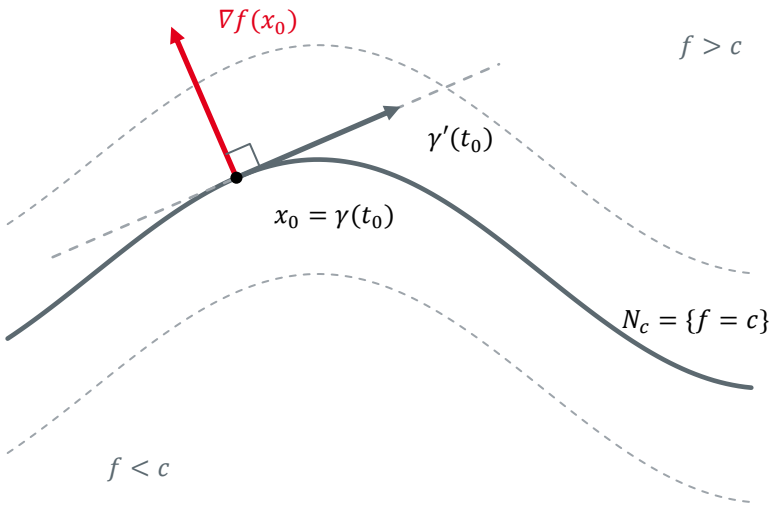


Abb. 3: Kurve γ in N_c mit Tangentenvektor $\gamma'(t_0)$ und Gradient $\nabla f(x_0)$; gestrichelt: Nachbarniveaus

Beweis

1 Hilfsfunktion: $g(t) = f(\gamma(t)), \quad t \in I$

2 Da γ in N_c verläuft, ist $g(t) = c$ konstant, also $g'(t) = 0$ für alle $t \in I$.

3 Kettenregel: $g'(t) = \nabla f(\gamma(t)) \cdot \gamma'(t) = \sum_{i=1}^{n} \frac{\partial f}{\partial x_i}(\gamma(t)) \cdot {\gamma_i}'(t)$

4 Einsetzen von $t = t_0$: $0 = g'(t_0) = \nabla f(x_0) \cdot \gamma'(t_0)$ ■

Warum die Voraussetzung $\nabla f(x_0) \neq 0$?

Dann ist N_c nach dem Satz über implizite Funktionen in der Nähe von x_0 eine glatte Kurve ($n = 2$) bzw. Fläche ($n = 3$), deren Tangentenvektoren genau die v mit $\nabla f(x_0) \cdot v = 0$ sind. Für $\nabla f(x_0) = 0$ kann N_c entarten: Bei $f = x^2 + y^2$ ist $N_0 = \{(0,0)\}$ nur ein Punkt, bei $f = xy$ besteht N_0 aus den beiden Achsen, die sich in $(0,0)$ kreuzen.

2. Der Gradient steht senkrecht auf der Niveaumenge

Aus dem Satz ergeben sich direkte Formeln für Tangenten an Höhenlinien und Tangentialebenen an Niveauflächen. Voraussetzung ist jeweils $\nabla f(x_0) \neq 0$.

Tangente an die Höhenlinie im Punkt (x_0, y_0) ($n = 2$)

$$f_x(x_0, y_0) \cdot (x - x_0) + f_y(x_0, y_0) \cdot (y - y_0) = 0$$

Ist zusätzlich $f_y(x_0, y_0) \neq 0$, so ist die Höhenlinie nahe x_0 ein Graph $y = y(x)$ mit der Steigung $y'(x_0) = -f_x(x_0, y_0)/f_y(x_0, y_0)$.

Tangentialebene an die Niveaufläche im Punkt x_0 ($n = 3$)

$$\nabla f(x_0) \cdot (x - x_0) = 0$$

Beispiel 1: $f(x, y) = x^2 + y^2$, $P = (3,4)$, $c = f(P) = 25$

$\nabla f(3,4) = (6,8)^{\mathrm{T}}$
Höhenlinie: Kreis vom Radius 5, $\gamma(t) = (5\cos t\,, 5\sin t)$
$P = \gamma(t_0)$ mit $\cos t_0 = 3/5$, $\sin t_0 = 4/5$
$\gamma'(t_0) = (-5\sin t_0\,, 5\cos t_0) = (-4,3)$
$\nabla f(P) \cdot \gamma'(t_0) = 6 \cdot (-4) + 8 \cdot 3 = 0$ ✓
Tangente: $6(x - 3) + 8(y - 4) = 0 \;\Leftrightarrow\; 3x + 4y = 25$

Beispiel 2: $f(x, y, z) = x^2 + y^2 + z^2$, $P = (1,2,2)$, $c = f(P) = 9$

Niveaufläche: Kugel vom Radius 3 um den Ursprung
$\nabla f(1,2,2) = (2,4,4)^{\mathrm{T}} = 2 \cdot (1,2,2)^{\mathrm{T}}$ (parallel zum Ortsvektor, also radial)
Tangentialebene: $2(x - 1) + 4(y - 2) + 4(z - 2) = 0 \;\Leftrightarrow\; x + 2y + 2z = 9$

Merkregel

Entlang der Niveaumenge ändert sich f nicht, quer dazu am stärksten. Der Gradient ist der Normalenvektor der Niveaumenge, die Tangenten bilden sein orthogonales Komplement:

$$\nabla f(x_0) \perp N_c \qquad \text{Tangentenrichtungen in } x_0\text{: } \{v \in \mathbb{R}^n : \nabla f(x_0) \cdot v = 0\}$$

3. Richtung des steilsten Anstiegs

Die Richtungsableitung von f in x_0 in Richtung eines Einheitsvektors v ($\|v\| = 1$) misst, wie schnell sich f ändert, wenn man von x_0 aus in Richtung v geht. Formal ist sie definiert als

$$D_v f(x_0) = \lim_{t \to 0} \frac{f(x_0 + tv) - f(x_0)}{t}$$

Andere Schreibweisen: $\partial_v f(x_0)$ oder $\partial f / \partial v\,(x_0)$. Für $f \in C^1(D)$ gilt

$$D_v f(x_0) = \nabla f(x_0) \cdot v = \|\nabla f(x_0)\| \cdot \cos\alpha$$

Herleitung: Auf der Geraden durch x_0 in Richtung v sei $g(t) = f(x_0 + tv)$. Dann ist $D_v f(x_0) = g'(0)$, und die Kettenregel liefert

$$g'(0) = f_x(x_0) \cdot v_1 + f_y(x_0) \cdot v_2 = \nabla f(x_0) \cdot v$$

Dabei ist α der Winkel zwischen $\nabla f(x_0) \neq 0$ und v; der zweite Schritt ist die Winkelformel des Skalarprodukts mit $\|v\| = 1$. Anschaulich ist das die Länge der Projektion von $\nabla f(x_0)$ auf v (rot in Abb. 4). Wegen $-1 \leq \cos\alpha \leq 1$ folgt sofort:

1 $\alpha = 0$: $D_v f(x_0) = \|\nabla f(x_0)\|$ ist maximal, v zeigt in Richtung $\nabla f(x_0)$: steilster Anstieg

2 $\alpha = \pi$: $D_v f(x_0) = -\|\nabla f(x_0)\|$ ist minimal, v zeigt in Richtung $-\nabla f(x_0)$: steilster Abstieg

3 $\alpha = \pi/2$: $D_v f(x_0) = 0$, keine Änderung: v ist tangential zur Höhenlinie

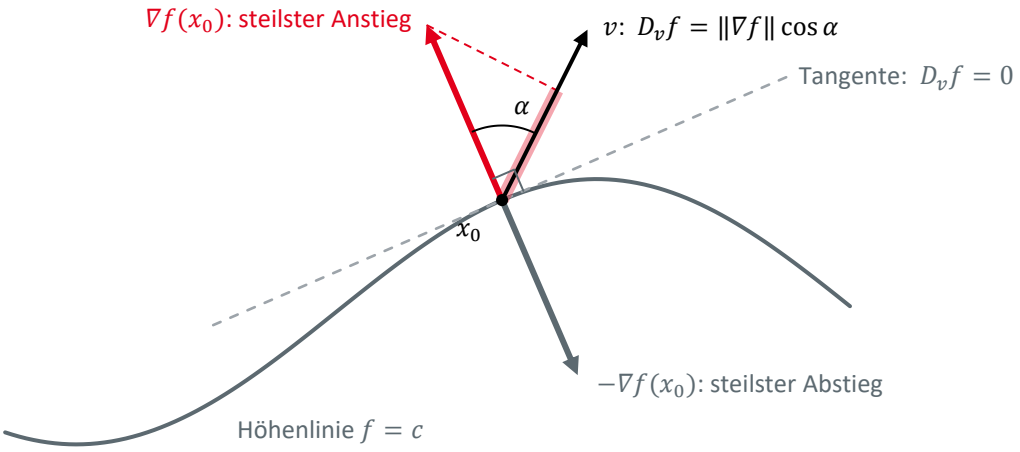


Abb. 4: Richtungsableitung in Abhängigkeit vom Winkel α zwischen v und $\nabla f(x_0)$

Punkt 3 liefert einen zweiten Zugang zum Satz, dass der Gradient senkrecht auf der Höhenlinie steht: Entlang der Höhenlinie ändert sich f nicht, die Richtungsableitung in Tangentenrichtung ist also null. Das ist nur für $\cos\alpha = 0$ möglich, die Tangente steht also senkrecht auf $\nabla f(x_0)$.

Ist $\nabla f(x_0) = 0$, so ist $D_v f(x_0) = 0$ für alle Richtungen v; x_0 heißt dann stationärer Punkt. Dort gibt es keine Richtung des steilsten Anstiegs.

4. Das Gradientenverfahren

Ziel: ein (lokales) Minimum von $f: \mathbb{R}^n \to \mathbb{R}$ numerisch finden. In Richtung $-\nabla f(x)$ fällt f am stärksten (steilster Abstieg). Das Gradientenverfahren geht deshalb wiederholt ein kleines Stück in diese Richtung:

$$x^{(k+1)} = x^{(k)} - \eta \nabla f(x^{(k)}), \qquad k = 0,1,2,\ldots$$

mit einem Startwert $x^{(0)}$ und einer Schrittweite $\eta > 0$ (beim maschinellen Lernen „Lernrate“ genannt).

Algorithmus

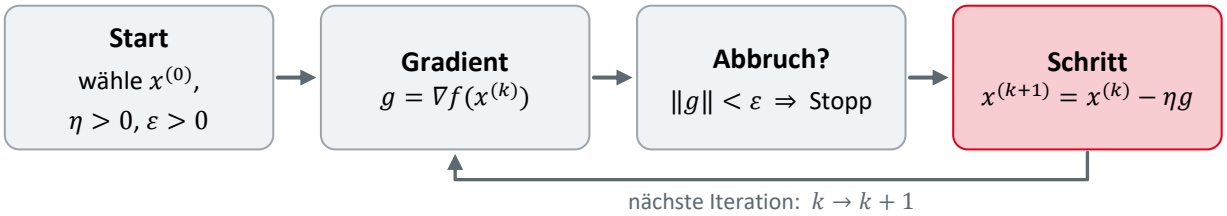


Abb. 5: Ablauf des Gradientenverfahrens

Warum sinkt f? Abstiegseigenschaft

Die Taylor-Entwicklung erster Ordnung $f(x+h) = f(x) + \nabla f(x) \cdot h + o(\|h\|)$ mit $h = -\eta \nabla f(x)$ liefert

$$f(x - \eta \nabla f(x)) = f(x) - \eta \|\nabla f(x)\|^2 + o(\eta) \qquad \text{für } \eta \to 0$$

Für $\nabla f(x) \neq 0$ ist der lineare Term negativ und überwiegt bei kleinem η den Rest. Für hinreichend kleines $\eta > 0$ gilt also $f(x^{(k+1)}) < f(x^{(k)})$. Zu große Schrittweiten können dagegen über das Minimum hinausschießen.

Zusammenhang mit Differentialgleichungen

Das Verfahren ist das Euler-Verfahren mit Schrittweite η für die DGL $x'(t) = -\nabla f(x(t))$ (Gradientenfluss). Entlang jeder Lösung gilt nach der Kettenregel

$$\frac{\mathrm{d}}{\mathrm{d}t} f(x(t)) = \nabla f(x(t)) \cdot x'(t) = -\|\nabla f(x(t))\|^2 \leq 0$$

f fällt also entlang der Lösungskurven, und diese schneiden die Höhenlinien senkrecht.

4. Das Gradientenverfahren

Beispiel: $f(x,y) = x^2 + 4y^2$, **Startwert** $x^{(0)} = (4,1)^{\mathrm{T}}$

Mit $\nabla f(x,y) = (2x, 8y)^{\mathrm{T}}$ lautet ein Schritt komponentenweise

$$x_{k+1} = x_k - \eta \cdot 2x_k = (1-2\eta)x_k \qquad y_{k+1} = y_k - \eta \cdot 8y_k = (1-8\eta)y_k$$

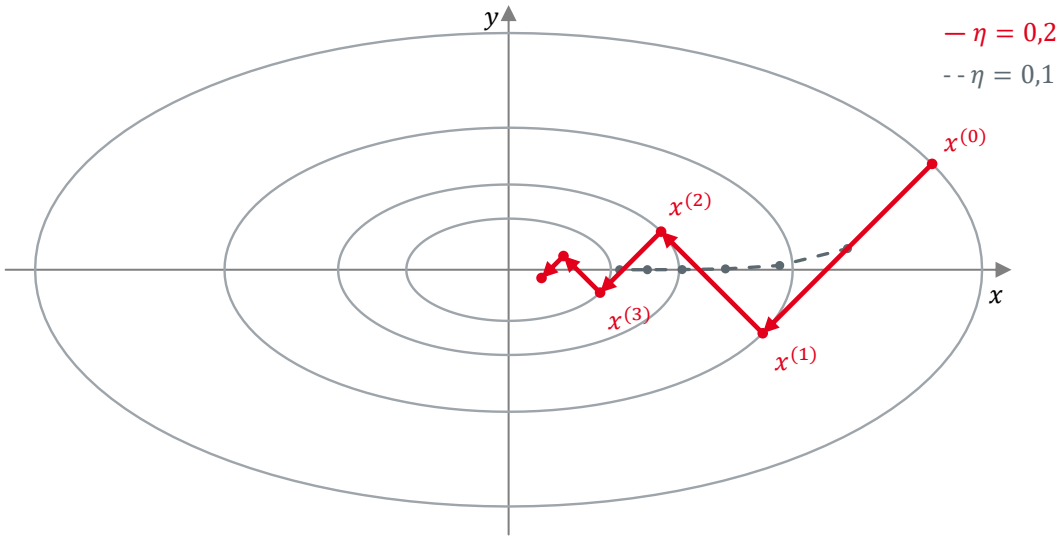


Abb. 6: Iterationen für $\eta = 0{,}2$ (rot) und $\eta = 0{,}1$ (gestrichelt); grau: Höhenlinien durch $x^{(0)}, \ldots, x^{(3)}$ für $\eta = 0{,}2$

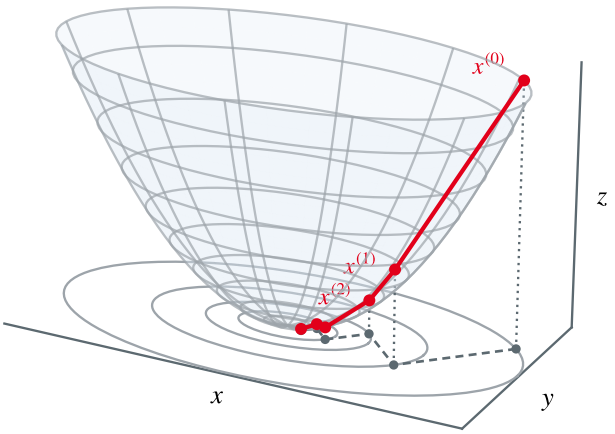


Abb. 7: Derselbe Weg ($\eta = 0{,}2$) auf der Fläche $z = f(x,y)$; gestrichelt: Schatten in der xy-Ebene

– Jeder Schritt verlässt $x^{(k)}$ senkrecht zur Höhenlinie durch $x^{(k)}$.

– $\eta = 0{,}2$: Die y-Komponente wechselt das Vorzeichen (Faktor $-0{,}6$), der Weg läuft im Zickzack. Wegen $f(x^{(k+1)}) = 0{,}36 \cdot f(x^{(k)})$ fällt f in jedem Schritt um 64 %.

– $\eta = 0{,}1$: kein Zickzack, aber in x-Richtung nur Faktor $0{,}8$; die Konvergenz ist langsamer.

4. Das Gradientenverfahren

Umsetzung in Python

Die Funktion setzt den Algorithmus direkt um: Gradient berechnen, Abbruch prüfen, Schritt ausführen. Zusätzlich wird der Weg gespeichert, damit man ihn anschließend zeichnen kann.

```
import numpy as np

def gradientenverfahren(grad, x0, eta, eps=1e-6, maxit=1000):
    """Minimiert f mit x(k+1) = x(k) - eta * grad f(x(k))."""
    x = np.array(x0, dtype=float)
    weg = [x.copy()]                    # Iterierte speichern
    for k in range(maxit):
        g = grad(x)                     # Gradient in x(k)
        if np.linalg.norm(g) < eps:     # Abbruch: |g| < eps
            break
        x = x - eta * g                 # Schritt gegen grad f
        weg.append(x.copy())
    return x, np.array(weg)

f    = lambda x: x[0]**2 + 4*x[1]**2
grad = lambda x: np.array([2*x[0], 8*x[1]])

x, weg = gradientenverfahren(grad, [4.0, 1.0], eta=0.2)
for k, p in enumerate(weg[:4]):
    print(f"k={k}: x=({p[0]:7.4f}, {p[1]:7.4f})  f={f(p):8.4f}")
print(f"{len(weg)-1} Iterationen, x=({x[0]:.1e}, {x[1]:.1e})")
```

Ausgabe

```
k=0: x=( 4.0000,  1.0000)  f= 20.0000
k=1: x=( 2.4000, -0.6000)  f=  7.2000
k=2: x=( 1.4400,  0.3600)  f=  2.5920
k=3: x=( 0.8640, -0.2160)  f=  0.9331
32 Iterationen, x=(3.2e-07, 8.0e-08)
```

Vergleich der Schrittweiten (mit eta=0.2 bzw. eta=0.1)

k	$x^{(k)}$ $(\eta = 0{,}2)$	$f(x^{(k)})$	$x^{(k)}$ $(\eta = 0{,}1)$	$f(x^{(k)})$
0	$(4; 1)$	20	$(4; 1)$	20
1	$(2{,}4; -0{,}6)$	7,2	$(3{,}2; 0{,}2)$	10,4
2	$(1{,}44; 0{,}36)$	2,592	$(2{,}56; 0{,}04)$	6,56
3	$(0{,}864; -0{,}216)$	0,9331	$(2{,}048; 0{,}008)$	4,1946
4	$(0{,}5184; 0{,}1296)$	0,3359	$(1{,}6384; 0{,}0016)$	2,6844

Mit $\eta = 0{,}1$ braucht das Verfahren 72 statt 32 Iterationen bis $\|\nabla f\| < 10^{-6}$.

Code zum Herunterladen (Repository llm-hausgebrauch auf GitHub)

Python-Skripte: zusatzmaterial/gradient_niveaulinien

Notebook mit Neuron-Beispiel: notebooks/gradientenverfahren_niveaulinien.ipynb

5. Wahl der Schrittweite

Am Beispiel $f(x) = x^2$ mit Minimum $x^* = 0$ lässt sich der Einfluss von η exakt angeben. Aus $f'(x) = 2x$ folgt

$$x_{k+1} = x_k - \eta \cdot 2x_k = (1 - 2\eta)x_k \quad \Rightarrow \quad x_k = (1 - 2\eta)^k x_0$$

1 $0 < \eta < 1/2$: Faktor in $(0,1)$, monotone Konvergenz gegen 0

2 $1/2 < \eta < 1$: Faktor in $(-1,0)$, Konvergenz mit Vorzeichenwechsel (das Verfahren schießt über)

3 $\eta > 1$: $|1 - 2\eta| > 1$, die Iterierten wachsen betragsmäßig, das Verfahren divergiert

Grenzfälle: Für $\eta = 1/2$ wird das Minimum in einem Schritt erreicht, für $\eta = 1$ pendelt das Verfahren zwischen x_0 und $-x_0$.

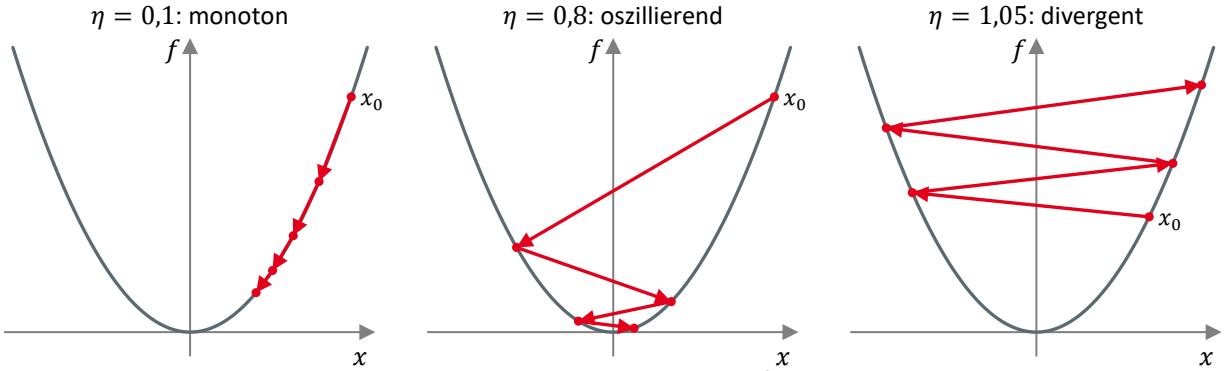


Abb. 8: Je vier Schritte des Gradientenverfahrens für $f(x) = x^2$ mit unterschiedlicher Schrittweite

Übertragung auf $f(x, y) = x^2 + 4y^2$

Beide Komponenten müssen konvergieren: $|1 - 2\eta| < 1$ und $|1 - 8\eta| < 1$, also $0 < \eta < 1/4$. Die stärkste Krümmung (Faktor 8 in y-Richtung) begrenzt die Schrittweite, die schwächste (Faktor 2 in x-Richtung) bestimmt das Tempo. Die beste konstante Schrittweite gleicht beide Faktoren aus:

$$1 - 2\eta = -(1 - 8\eta) \quad \Rightarrow \quad \eta = 0{,}2, \quad \text{Kontraktionsfaktor } 0{,}6 \text{ pro Schritt}$$

Allgemein gilt für $f(x) = \tfrac{1}{2}x^{\mathrm{T}}Ax$ mit symmetrischer, positiv definiter Matrix A und Eigenwerten $\lambda_{\min} \le \cdots \le \lambda_{\max}$: Das Verfahren konvergiert für jeden Startwert genau dann, wenn $0 < \eta < 2/\lambda_{\max}$. Optimal ist $\eta = 2/(\lambda_{\min} + \lambda_{\max})$ mit Faktor $(\kappa - 1)/(\kappa + 1)$, $\kappa = \lambda_{\max}/\lambda_{\min}$. Hier ist $A = \mathrm{diag}(2,8)$, also $\kappa = 4$ und Faktor $3/5$.

6. Übungsaufgaben

Aufgabe 6.1: Gradient und Höhenlinie

$$f(x,y) = x^2 + 4y^2, \quad P = (2,1)$$

a) Bestimmen Sie das Niveau c der Höhenlinie durch P und beschreiben Sie diese Höhenlinie.

b) Berechnen Sie $\nabla f(P)$ und die Gleichung der Tangente an die Höhenlinie in P.

c) Die Höhenlinie wird durch $\gamma(t) = \left(2\sqrt{2}\cos t\,, \sqrt{2}\sin t\right)$ parametrisiert. Bestimmen Sie t_0 mit $\gamma(t_0) = P$ und zeigen Sie $\nabla f(P) \cdot \gamma'(t_0) = 0$.

Aufgabe 6.2: Richtung des steilsten Anstiegs

$$f(x,y) = x \cdot e^{y}, \qquad P = (2,0)$$

a) Berechnen Sie $\nabla f(P)$. In welche Richtung wächst f in P am stärksten, und wie groß ist diese maximale Änderungsrate?

b) Berechnen Sie die Richtungsableitung $D_v f(P)$ für $v = (3,4)^{\mathrm{T}}/5$ und den Winkel zwischen v und $\nabla f(P)$.

c) Bestimmen Sie alle Einheitsvektoren v mit $D_v f(P) = 0$. Zeigen Sie, dass sie tangential zur Höhenlinie durch P liegen; lösen Sie dazu die Gleichung der Höhenlinie nach y auf.

Aufgabe 6.3: Gradientenverfahren von Hand

$$f(x, y) = (x - 1)^2 + 2(y + 2)^2$$

a) Berechnen Sie ∇f und geben Sie das Minimum von f ohne Rechnung an.

b) Führen Sie ausgehend von $x^{(0)} = (0{,}0)^{\mathrm{T}}$ zwei Schritte des Gradientenverfahrens mit $\eta = 0{,}2$ aus und berechnen Sie jeweils $f(x^{(k)})$.

c) Für welche Schrittweiten $\eta > 0$ konvergiert das Verfahren für jeden Startwert? Welche konstante Schrittweite ist optimal?

Aufgabe 6.4: Gradientenverfahren in Python

$$f(x,y) = x^2 + 4y^2, \quad x^{(0)} = (4,1)^{\mathrm{T}}, \quad \varepsilon = 10^{-6}$$

a) Erweitern Sie die Python-Funktion gradientenverfahren aus dem Vortrag: Sie soll abbrechen, sobald $\|x^{(k)}\| > 10^6$ (Divergenz), und zusätzlich die Anzahl der Iterationen zurückgeben.

b) Bestimmen Sie die Anzahl der Iterationen für $\eta = 0{,}05; 0{,}1; 0{,}2; 0{,}24; 0{,}26$.

c) Zeichnen Sie die Wege für $\eta = 0{,}2$ und $\eta = 0{,}24$ in ein Höhenlinienbild (matplotlib, contour) und erklären Sie die Ergebnisse aus b).

Aufgabe 6.5: Ein Neuron trainieren

$$\hat{y} = \sigma(wx + b), \quad \sigma(z) = 1/(1 + e^{-z}), \quad \sigma' = \sigma(1 - \sigma), \quad L = (\hat{y} - y)^2$$
Trainingsbeispiel $(x, y) = (2,0)$, Startwerte $w = b = 0$

a) Vorwärtsrechnung: Berechnen Sie $z = wx + b$, $\hat{y}$ und den Fehler L.

b) Rückwärtsrechnung: Berechnen Sie mit der Kettenregel $\partial L/\partial \hat{y}$, $\partial \hat{y}/\partial z$ und daraus $\partial L/\partial w$ und $\partial L/\partial b$.

c) Führen Sie einen Schritt des Gradientenverfahrens mit $\eta = 1$ aus. Wie groß ist der Fehler mit den neuen Parametern?

d) Für dieses eine Trainingsbeispiel hängt L nur von $z = 2w + b$ ab. Beschreiben Sie die Höhenlinien von $L(w, b)$ und zeigen Sie, dass $\nabla L(w, b)$ senkrecht auf ihnen steht.

7. Ausblick: Gradientenverfahren in der KI

Beim Training eines KI-Modells, etwa eines neuronalen Netzes, werden die Parameter $\theta \in \mathbb{R}^p$ (Gewichte und Verschiebungen) so bestimmt, dass eine Verlustfunktion $L(\theta)$ klein wird. L misst den Fehler des Modells auf den Trainingsdaten. Große Sprachmodelle haben Milliarden von Parametern; das Grundprinzip ist trotzdem das Gradientenverfahren:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla L(\theta^{(k)}) \qquad (\eta\text{: Lernrate})$$

Bei der linearen Regression $y \approx wx + b$ ist $L(w,b) = (1/m)\sum_{i=1}^{m}(wx_i + b - y_i)^2$ eine quadratische Funktion wie in den Beispielen oben. Bei neuronalen Netzen ist L eine Verkettung vieler Funktionen, und den Gradienten liefert die Kettenregel.

Ein Neuron und die Backpropagation

Ein Neuron berechnet $\hat{y} = \sigma(wx + b)$ mit der Sigmoidfunktion $\sigma(z) = 1/(1 + e^{-z})$, für die $\sigma' = \sigma(1 - \sigma)$ gilt. Für ein Trainingsbeispiel (x, y) und den Fehler $L = (\hat{y} - y)^2$ liefert die Kettenregel

$$\partial L/\partial w = 2(\hat{y} - y) \cdot \sigma'(z) \cdot x \qquad \partial L/\partial b = 2(\hat{y} - y) \cdot \sigma'(z)$$

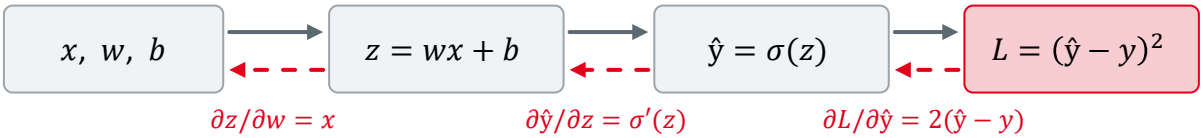


Abb. 9: Vorwärtsrechnung (grau) und Rückwärtsrechnung der Ableitungen mit der Kettenregel (rot, gestrichelt)

Zahlenbeispiel

$x = 1,\ y = 1,$ Start $w = b = 0,\ \eta = 1$: $z = 0,\ \hat{y} = 0{,}5,\ L = 0{,}25$

$\partial L/\partial w = \partial L/\partial b = 2 \cdot (-0{,}5) \cdot 0{,}25 = -0{,}25 \ \Rightarrow\ w = b = 0{,}25$

Neuer Fehler: $z = 0{,}5,\ \hat{y} \approx 0{,}6225,\ L \approx 0{,}1425$ (kleiner als 0,25)

In einem tiefen Netz wird diese Kettenregel Schicht für Schicht von hinten nach vorn angewandt (Backpropagation). So erhält man alle p partiellen Ableitungen mit einem Aufwand in der Größenordnung einer einzigen Vorwärtsrechnung.

7. Ausblick: Gradientenverfahren in der KI

Varianten in der Praxis

– **Stochastisches Gradientenverfahren (SGD):** ∇L wird nur aus einem kleinen, zufällig gewählten Teil der Daten (Mini-Batch) geschätzt. Jeder Schritt ist billig, die Richtung dafür verrauscht.

– **Momentum:** Die Schrittrichtung mittelt über die vergangenen Gradienten: $v \leftarrow \beta v + \nabla L(\theta)$, $\theta \leftarrow \theta - \eta v$ mit $0 < \beta < 1$. Das dämpft das Zickzack aus Abb. 6.

– **Adam:** passt die Schrittweite für jeden Parameter einzeln an die Größe seiner bisherigen Gradienten an. Adam ist ein Standardverfahren beim Training großer Modelle.

– **Lernraten-Plan:** η wird im Lauf des Trainings verändert, typischerweise erst langsam erhöht (Warm-up) und dann schrittweise verkleinert.

Grenzen des Verfahrens

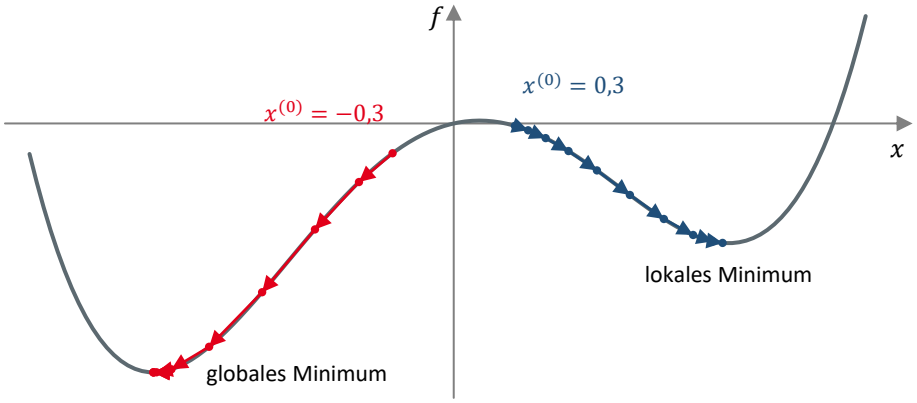


Abb. 10: $f(x) = x^4 - 4x^2 + x, \eta = 0{,}05$: Benachbarte Startwerte führen zu verschiedenen Minima

– **Lokale Minima:** Bei passender Schrittweite landet das Verfahren meist in einem Minimum nahe beim Startwert, nicht notwendig im globalen Minimum (Abb. 10). *Abhilfe:* mehrere Startwerte ausprobieren; das Rauschen von SGD und das Momentum helfen, flache Mulden wieder zu verlassen.

– **Sattelpunkte:** Dort ist $\nabla f = 0$ ohne Extremum, z. B. $f(x, y) = x^2 - y^2$ in $(0{,}0)$. Ob ein Minimum vorliegt, entscheidet die Hesse-Matrix. *Abhilfe:* Ein Sattelpunkt ist instabil. Schon eine kleine Störung oder das Rauschen von SGD führt weiter abwärts.

– **Schlechte Kondition:** Langgezogene Höhenlinien erzwingen kleine Schrittweiten und erzeugen Zickzack. *Abhilfe:* Variablen so skalieren, dass die Höhenlinien runder werden (Vorkonditionierung), oder Momentum bzw. Adam verwenden. Das Newton-Verfahren nutzt die Hesse-Matrix direkt.

Die Verlustfunktionen neuronaler Netze haben sehr viele Variablen und viele lokale Minima und Sattelpunkte. Dass das einfache Gradientenverfahren dort trotzdem gute Modelle findet, ist Gegenstand aktueller Forschung.